

GaN Motor Driver

Introduction

This report details my process for developing a custom gallium nitride (GaN) field-effect transistor (FET) motor driver. The motivation for this project was simply to learn. Though I am drawn to electromechanical systems, which a motor driver is a quintessential example of, I had a surface-level understanding of how a motor driver works and wanted to dig deeper. I also wanted an apt project through which I could learn STM32 development. I had additionally very nearly completed a similar project in Spring 2025 for Northwestern's Robot Design Studio class, and I was unsatisfied with the result and sought to finish what I had started. Given I only had one quarter to complete this project, I relied on some of the materials from the previous year's effort and clearly defined the project scope.

The goal: Develop a brushless DC (BLDC) motor driver capable of...

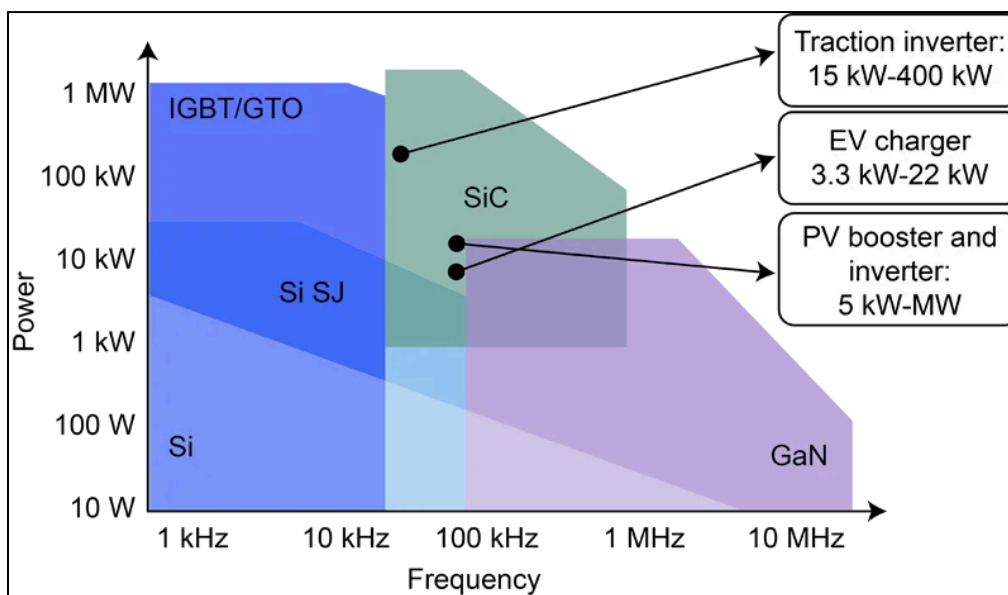
- 48 V DC supply
- 53 A phase current
- Current and voltage measurements to enable popular control algorithms
- Running at a very fast frequency
- Debug-friendly design
- Overcurrent protection

The main work of this project consisted of designing a single PCB that integrated the above features. A separate MCU development kit, an STM32 Nucleo board, was used to drive the custom PCB and read its output signals. The use of a pre-made development board decoupled the core low-level motor driver concepts, which were my primary concern to learn, from the potential pitfalls of integrating a bare MCU chip onto a custom PCB – no need for properly integrating a crystal, USB connectivity, etc. on a custom design. The remainder of this report will detail the specifications of the custom board, the details of my bringup process for both the hardware and software, and what the end product is capable of.

Note: *This is not groundbreaking work! I did not undertake this project to advance the fields of power electronics or motor drives, nor experiment with even slightly advanced motor control algorithms. This was a personal challenge to learn how a modern motor driver works by making one myself – not one that's bleeding edge, but one that is comparable to modern options by using modern components.*

Prelude: Why GaN?

This project has its roots in a previous year's project developed for the Human Augmentation via Dexterity Engineering Research Center (HAND ERC) through Northwestern's Robot Design Studio class. In robotics, and especially for robot dexterity, control bandwidth is very important. Higher control bandwidth means faster responses and more natural robotic motion. The control bandwidth, i.e. how fast you can effect changes in the system, is limited by the rate at which your power electronics switch. Though I wasn't writing the control algorithms, as the EE in charge of the driver hardware, I sought to raise the ceiling for control speed as high as I reasonably could. This is where GaN, a relatively new wide bandgap semiconductor, comes in. GaN FETs have extremely low parasitic capacitance, making them capable of switching at extremely high speeds. They also come with impressively low on-resistance, meaning they can carry higher transient currents compared to traditional silicon (Si) FETs for the same transistor channel width. However, in the current state of the art, GaN FETs' thermal conductivity is slightly lower than that of Si FETs and significantly lower than that of silicon carbide (SiC), another wide bandgap semiconductor that has found its way in to many automobiles and advanced, high power density electric drives. These qualities make GaN a perfect niche for robotics: relatively low continuous power, yet still able to handle high transient currents, and capable of raising the ceiling for control algorithms by switching extremely fast. A comparison of the aforementioned three semiconductors (plus IGBTs, not relevant here) is shown below.



A comparison of IGBTs and Si, SiC, and GaN FETs. [Source](#)

While the potential for high switching speed is beneficial for control bandwidth, it also presents a problem: it puts pressure on the gate driver design to have very low inductance and not introduce stray capacitance that would nullify the benefits of using GaN in the first place, or otherwise produce nasty ringing that would need careful tuning. Either way, developing such a gate driver would be expensive and time-intensive, so I chose last year to rely on a chip that tightly integrated

an entire half-bridge gate driver with high- and low-side GaN FETs: The [LMG2100R044](#). This is a TI device capable of 100 V supply and 35 A phase current, and supports logic-level inputs to close the FETs. It is an amazing creation, but not without a hefty pricetag of [\\$9.73 on DigiKey](#) apiece in low quantities (as of June 2026). With the same tight timeline and similar goals for this year (i.e. learn about how to drive a motor, not design a gate driver), I intended to use the same chip. Upon visiting TI's website, I discovered an updated version using slightly wider GaN FETs, the [LMG2100R026](#), which is capable of the same 100 V supply but can handle an increased 53 A continuously. I did not see a reason to limit phase current by the selection of chip, so I decided to go with the newer version at a couple dollars more apiece ([\\$13 on DigiKey](#) as of June 2026).

Requirements

Voltage: 48 V

For this motor driver to be usable, it should be built to be used with modern motors. 48 V was the highest voltage I was comfortable selecting, and most of the hobby motor world agrees – increasing your voltage capability to 48 V greatly increases your selection of high-powered motors. I found that increasing much more than 48 V did not increase selection size very much, at least up to the 100 V limit of the chip.

Phase current: 53 A

The phase current rating comes directly from the chip, as I saw no reason to limit what the chip was capable of. Besides possible trace heating for long durations of high-torque operation, everything on the board should be capable of delivering, sensing, and operating with 53 A entering or leaving any phase.

PWM frequency: ~200 kHz

This was a more vague goal, as it was less defined by hardware, and I have little experience with designing control algorithms. I at least wanted to aim higher than the traditional 10-20 kHz that almost every other commercial motor driver runs at, and I thought a 10x upgrade was worth a shot. Quite a bit of copper on the board contains signals at the PWM switching frequency, so much care was taken in layout to reduce the effects of crosstalk and both common-mode and differential-mode interference, especially on sensitive sensing lines.

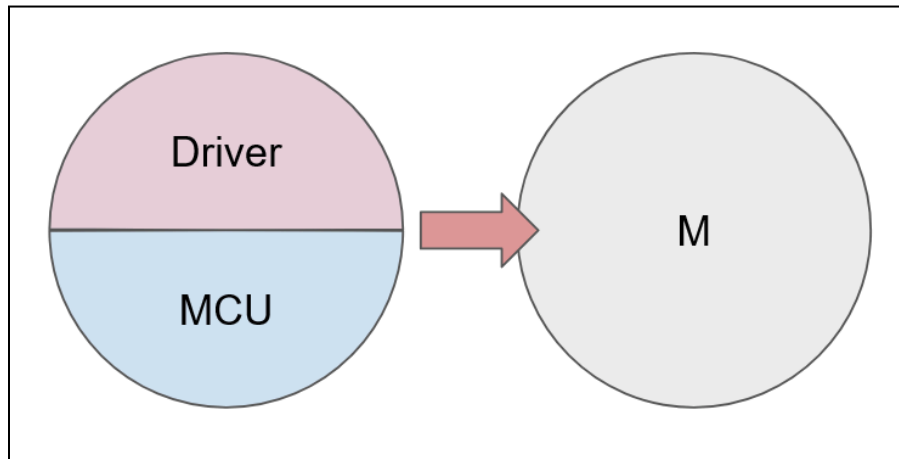
Debug-friendly

This comes as a direct lesson from the previous project. Mechanical constraints were stringent enough for the prototype robotic hand in RDS 2025 that I created a very tight layout for that first iteration of GaN motor driver, which was actually a dual motor driver with 6 individual phases. It also integrated an additional stage of circuitry I called deadtime control, which enforced deadtime in hardware in case the software attempted to close both FETs in a half-bridge at once (something the LMG2100 chips do not internally prevent). Without the requirements of a larger project to meet, I was free to design this project to be much more obviously a prototype: lower-density component layout, capable of driving only a single motor, fewer components (deleted deadtime control), and

Bill of materials for exact quantities used on board.

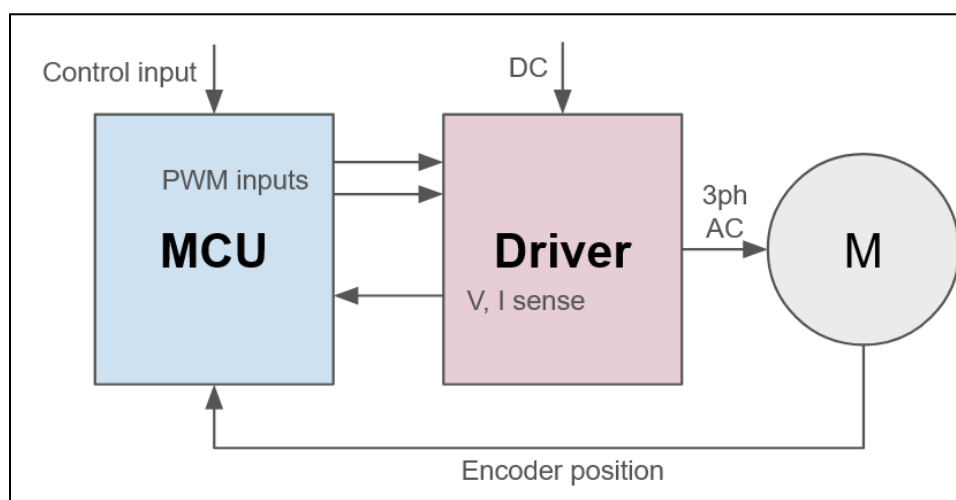
Architecture

The long-term vision I have for this project is an *integrated motor drive* (see [H3X](#)). I have no hope of making anything a tenth as power dense as one of their motors, but I am very drawn to the idea of closely integrating the mechanical and electrical components of a motor into a nice neat package: DC power and control signal go in, torque comes out. Such a project would involve integrating the control side with the driver side onto a single PCB capable of mounting directly to a motor, as shown below.



The long-term goal for this project: co-package the control and drive electronics with the motor.

I wanted a stepping stone before jumping to such a constrained design, so I set my sights on a simpler architecture first: a dedicated driver board handling the fast-switching, high-magnitude voltages and currents a safe distance away from an off-the-shelf MCU development kit. A system overview is shown in the block diagram below.



Simplified system block diagram. The red Driver block is what this report details.

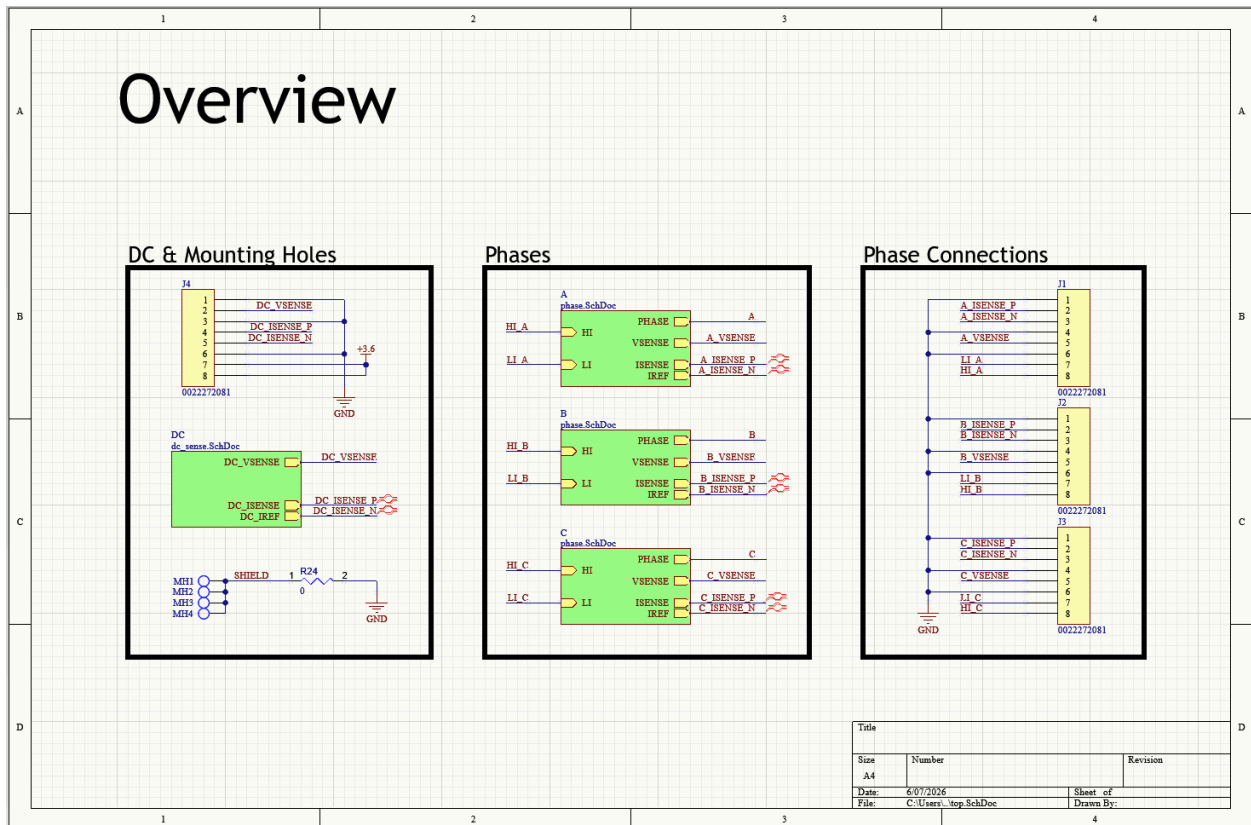
The blue MCU block is the STM32 Nucleo board, the red Driver block is the custom PCB, and the grey M circle is a 3-phase BLDC motor. All of the early work on this project focused on the driver board: its schematic design, PCB layout, and bringup. It features inline shunt current sensing, direct voltage sensing, optional 3/6PWM input buffers, overcurrent protection, header-friendly signal connectors, simple solder pads for the high-current connections, an E-field shielding inner layer, and finally, awesome hidden graphics (you'll see). All of these features are described or shown in detail in the following two sections.

Hardware

Schematic

Below are the complete schematic sheets. They can be viewed in higher resolution [here](#).

Overview

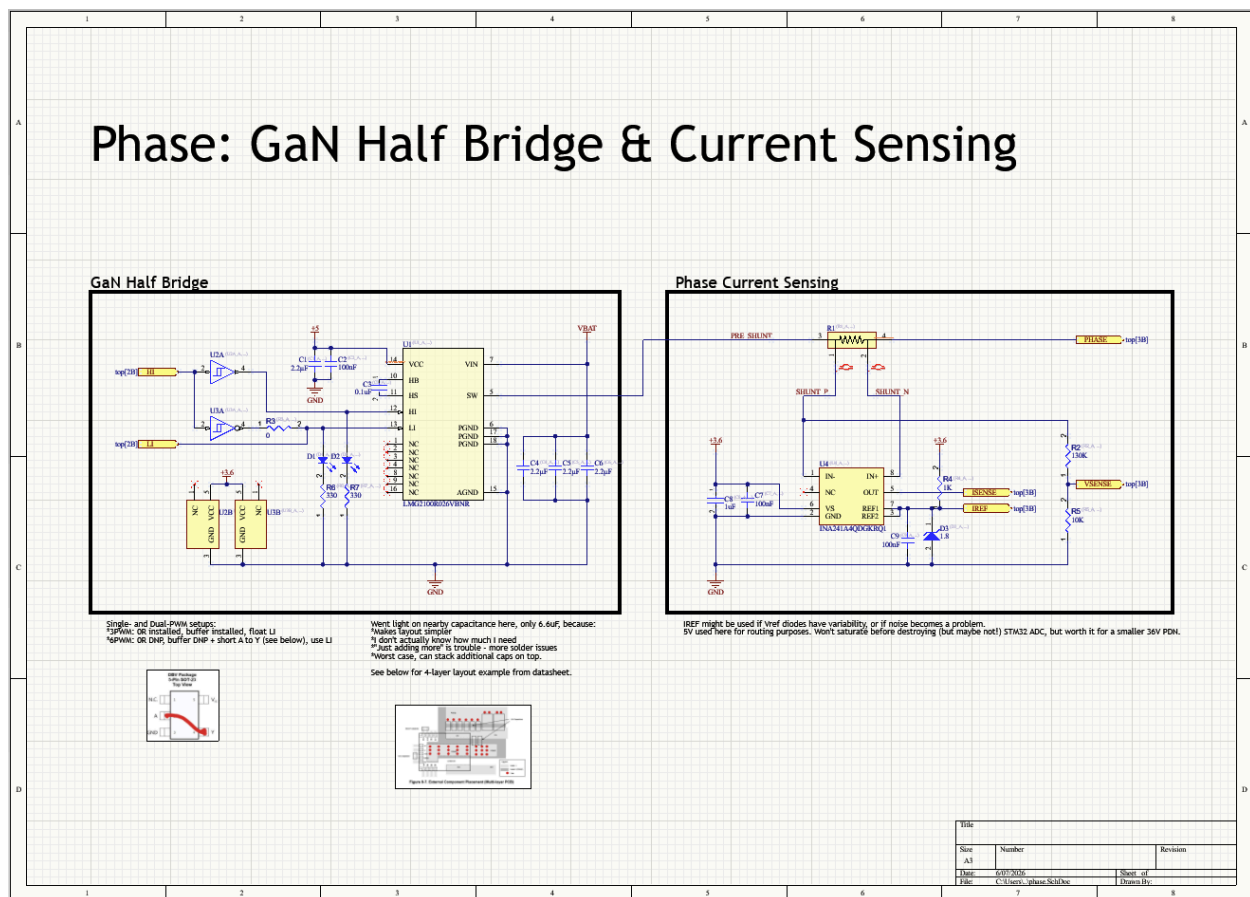


Overview schematic sheet.

The overview sheet includes the four signal connectors, three sets of phase circuitry, the four mounting holes and grounding, and the remainder of the board's components under the "DC" block.

- Green blocks represent other schematic sheets, and the inlaid yellow arrows show their connectivity. You will see the corresponding yellow arrows on the following schematic sheets.
- The decision to use four signal connectors came from a desire to have three identical connectors, one for each phase, and the remainder of the signals (loosely referred to as "DC") were grouped into a fourth. They neatly fit on one side of the board within dimensions I was comfortable with.
- The four mounting holes are tied to GND via 0R jumper R24 because they connect to a specific inner shielding layer. Directly assigning the GND net to the mounting holes and shield layer would have caused all four mounting holes, as well as many GND vias (!) to be directly connected to the shield. Conventional EMC wisdom states that shielding of that kind should be connected to ground in exactly one place to prevent ground loops and generally improve ground quality; the way I achieved this sort of connectivity in Altium without making a thousand design rules was by using a jumper to separate the two nets – I was quite happy with this solution, provided I had the available board real estate.

Phase

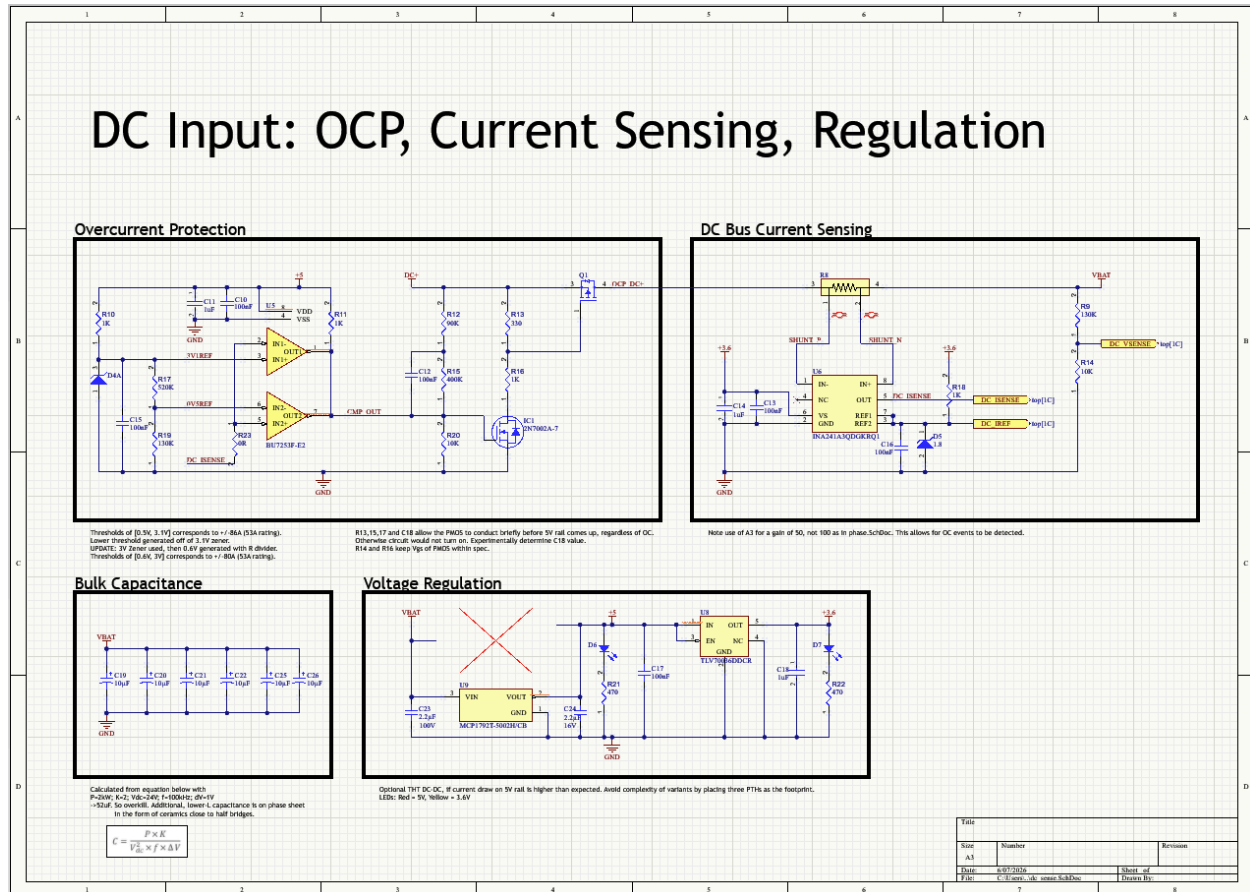


The phase sheet is where the most "motor driver" components lie: The half-bridge module and surrounding circuitry, plus the inline shunt current sensing.

- The half-bridge chip integrates so much that about the only thing needed to support it are capacitors. All of them were deliberately chosen, particularly their construction, value, and package:
 - C1 and C2 are ceramic decoupling capacitors to filter noise of differing frequencies off of the 5 V rail, which supplies the LMG2100's internal circuitry (UVLO, level shifting, etc.).
 - C3 is the bootstrap capacitor. TI explicitly recommends a ceramic 0.1 uF 0603 for this purpose, and to closely locate it to the HS and HB pins of the chip. Note that without an isolated supply like you might see in higher-power motor drivers, this capacitor is what supplies the gate of the high-side FET with sufficiently high voltage to switch DC+ to the switch node. See page 11 of the [LMG2100 datasheet](#).
 - C4, C5, and C6 are 2.2 uF 0805 ceramic capacitors that provide some of the bulk capacitance that is recommended for supplying current in high transients. These have lower parasitic inductance than the wound aluminum electrolytics shown on the next sheet, which makes them great at handling higher frequency transients.
- A buffer and inverting buffer are fed by the HI input.. They allow what is called a "3PWM" configuration, where only three PWM outputs are needed to drive the half-bridges. This saves pins on the MCU at the cost of a fixed dead time and no option to completely open each half-bridge; 3PWM means that one switch of each half-bridge will always be closed. The alternative is 6PWM, which would be configured by depopulating jumper R3 and the buffer, then shorting input to output across the buffer's footprint as shown in the schematic sheet.
- LEDs were included on the HI and LI inputs, which are the logic-level inputs that close the high- and low-side switch, respectively. This was done for debugging, and proved very helpful as a reliable sanity check. Effects of added capacitance on these lines from the diodes are not exactly known, but assumed to be negligible at feasible PWM frequencies.
- The right side of the sheet shows the two major components of each phase's current sensing circuitry: the shunt resistor and the amplifier. The shunt resistor sits inline with the high current path so that the voltage it drops is readable as a differential voltage by a differential amplifier. This means it must be very low value to avoid wasting power. This then means that the amplifier must be high gain – it was chosen to be 100 (see section below for calculations). The amp must also be capable of rejecting common-mode voltages quite well, as the common-mode voltage the resistor sees wildly fluctuates with the switching of the half-bridge. More on that later.
- A zener diode supplies the reference voltage that the amplifier adds its reading to; a reference voltage greater than zero allows bidirectional current sensing. The value of 1.8 V was chosen to be right in the middle of the STM32 ADCs' max input voltage of 3.6 V, which is also what the amplifier gets supplied so that it may never exceed the STM's max rating. Being in the middle of the range maximizes measurement range of positive and negative current, which we expect to be symmetrical.

- The resistor inline with the zener diode was selected to be too low; the reference voltage is quite a bit higher than 1.8 V with a 1 kR.
- Voltage sensing is implemented with a simple resistor voltage divider.

DC Input



DC input schematic sheet.

The DC input sheet contains the rest of the board's circuitry: OCP, DC bus current sensing, bulk capacitance for the half-bridges, and voltage regulation.

- OCP is a doozy – see full section below.
- DC bus current sensing is largely the same as the phase current sensing, albeit with an amplifier with a gain of only 50, not 100. This allows for twice the range of current sensing, mainly for the purposes of sensing an overcurrent event. This hurts resolution, but DC current is usually not used by control algorithms that would benefit from accuracy.
- The bulk capacitance is made of high-valued aluminum electrolytic capacitors that take up the remainder that the 2.2 uF ceramics on the phase sheet don't cover. See layout for how these are distributed underneath the three half-bridges. See calculations in section below.
- Voltage regulation simply takes in the input voltage and regulates it down to 5 V and 3.6 V. 5 V is used exclusively for the comparators and the half-bridges, the latter of which require it

for their internal circuitry. 3.6 V supplies everything else: the buffers, current sense chips, zener diodes, and the STM32 board if not externally supplied.

Major decisions

Bulk capacitance

This equation for DC link capacitance (from [this article](#) on inverter design) was used to estimate the amount of bulk capacitance to place across the supply rails:

$$C = \frac{P \times K}{V_{dc} \times f \times \Delta V}$$

The meaning and values chosen for each parameter is given in the table below.

Parameter	Meaning	Value chosen	Justification
P	Power	2000 W	In the ballpark of 48 V * 53 A = 2544 W
K	Voltage ripple tolerance, typically [1, 2]	2	Low voltage ripple tolerance; conservatively oversizes C
V _{DC}	Supply voltage	24 V	Though the board can support 48 V, this conservatively oversizes C
f	Switching frequency	100 kHz	Aiming for a higher figure, so this conservatively oversizes C
ΔV	Allowable voltage ripple	1 V	2% of 48 V

Given these parameters, the resulting bulk capacitance is 69.4 uF.

I assumed that each phase could make use of all of the large electrolytic capacitors, but only the close-by ceramic capacitors. Therefore, capacitors with the following parameters were chosen.

Capacitor type	Package	Value	Quantity	Total capacitance per phase
Electrolytic	Radial can	10 uF	6 total	60 uF
Ceramic	0805	2.2 uF	3 per phase	6.6 uF

This makes the equivalent total bulk capacitance available to each phase 66.6 uF, which is 96% of the calculated value of 69.4 uF. I deemed this good enough given my conservative sizing calculation.

Current sense

Inline phase current sense circuitry requires high-quality components. The differential amplifier has the most important job: Convert a small voltage across a resistor to a cleanly amplified output without including common-mode voltage, which wildly flies around because of how fast the half-bridge is being driven. The requirements for the current sense circuitry are:

- Sense at least 53 A
- Don't amplify noise
- Output remains within 0 to 3.6 V (STM32 ADC max input voltage)
- Not exceed shunt resistor's power rating

I decided to keep my choice of amplifier from last year's project, the INA241. It is a remarkably flexible family of devices: They are the state of the art in current sense amplifiers: they can take as high as 20V supply, have an extremely low common-mode rejection ratio (CMRR), come in a range of fixed gain options, and are easy to design around. Note that two different gain options were used for each of the phase current sense circuit and the DC current sense circuit: 100 and 50, respectively, to allow for overcurrent detection on the DC line. The "Current Sensing" page of [this spreadsheet](#) supported my search for a combination of gain choice and shunt resistor. I did not pay much mind to the power lost across the shunt resistor beyond whether or not it was within the device's rating. The final shunt resistor, the [Yageo PU2726](#), has the below specifications.

Shunt resistor spec	Nominal value
Value	0.3 mOhm
Power	11 W

The final spreadsheet after selection of the shunt resistor is shown below. It accounts for component variation and indicates (with a green highlight) when output parameters are desirable or within the given shunt ratings. All important parameters of the current sense circuit are shown.

	A	B	C	D	E	F	G	H	I	J	K	L	M
1													
2	Current sensing: INA241												
3					Notes								
4	Max phase current	53 A, +/-	Set	Set by GaN chip, 2.6mOhm now									
5	Shunt resistor value	0.3 mOhm	Variable										
6	Shunt resistor power rating	11 W	Variable										
7	Shunt tolerance	1 %											
8													
9	Shunt resistor max	0.303 mOhm	Worst-case calcs here only account for shunt R variation										
10	Shunt resistor min	0.297 mOhm											
11													
12	Max shunt power (nom)	0.8427 W	Output	Not even close to a problem. Deciding factor became low gain for using as much ADC range as possible.									
13	*max	0.8511 W	Output										
14	*min	0.8343 W	Output										
15	Shunt V at max I (nom)	0.0159 V	Output										
16	*max	0.0161 V	Output										
17	*min	0.0157 V	Output										
18													
19	INA241 supply V	3.6 V	Set										
20	INA241 V reference	1.8 V	Set										
21	INA241 gain	100 V/V	Variable	Options: 10, 20, 50, 100, 200									
22													
23	INA241 output swing (nom)	1.59 V, +/-	Output										
24	*max	1.6059											
25	INA241 max output (nom)	3.39											
26	*max	3.4059											
27	INA241 min output (nom)	0.21											
28	*min	0.1941											

Spreadsheet that aided in shunt resistor selection.

3PWM buffer and inverter

To allow for more flexible usage of MCU pins, I designed the board so that each phase could be driven with one or two PWM channels. The use of two PWM channels per phase is called 6PWM, and the use of one is called 3PWM. 6PWM allows the most control over the closing of each side of the half-bridge at the cost of three additional MCU pins. 3PWM simplifies pin usage by always closing one side of the half-bridge: a logic HIGH closes the upper switch, and a logic LOW closes the lower switch. This channel would directly drive the input for the upper switch, while its inverse would drive the input for the lower switch. Therefore, a digital inverter is used to invert the 3PWM channel to drive the lower switch. I added a buffer before the input to the upper switch to try to compensate for the delay added by the inverter. Close attention was paid to the propagation delays of these devices, since that parameter would 1) add latency to inputs and 2) cause problems if mismatched. The details of chosen two parts are given in the table below for room temperature operation at as close to 3.6 V supply as they list.

Part	Datasheet	t_{pd} (max, min) (ns)
SN74AUP1G17 (buffer)	Link	2.2, 5.7
SN74LVC1G14 (inv. buffer)	Link	1.8, 6.5

To use 3PWM, all devices in the half-bridge portion of the Phase schematic sheet are to be installed on the board, and the LI input is to be left floating. If 6PWM is to be used, the hardware can be reconfigured in the following way:

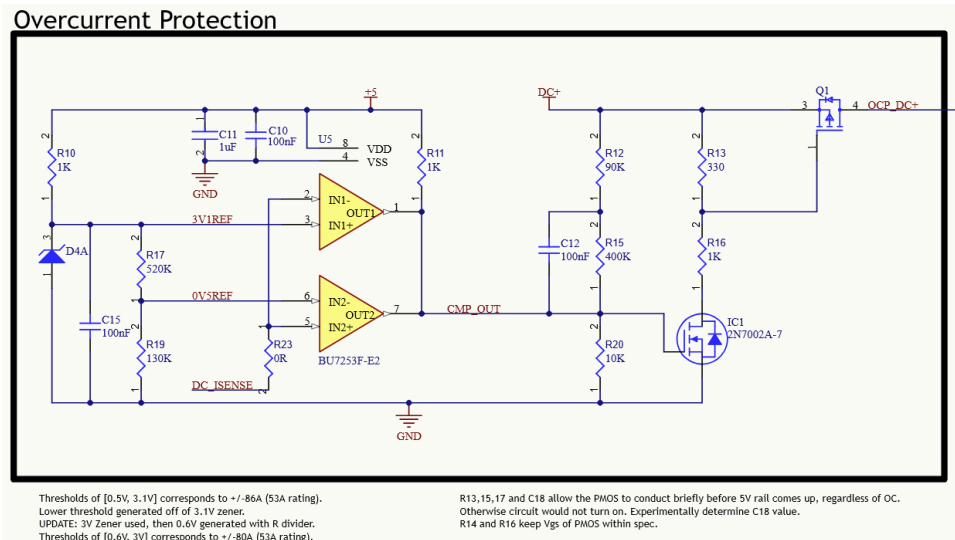
- Do not populate R3
- Do not populate buffer IC

- Short the input and output pads on the unpopulated buffer footprint

Since my pin budget allowed for it through all stages of testing, I opted to use 6PWM along with complementary PWM generation that the STM32 supports. This feature allows simple programming of dead time, which I would not be able to control with the simple design I chose to support 3PWM.

Overcurrent protection

A blow-up of the overcurrent protection circuit schematic is shown below.

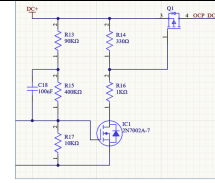


Overcurrent protection circuit.

This circuit simply opens a Si PMOS inline with the main supply line when it detects an overcurrent condition. Detection is done by running the output of the DC current sense circuit through a window comparator, which then drives a Si NMOS to switch the gate of the PMOS. A resistor network is used in the right-hand side of the schematic to meet V_{gs} ratings of both FETs, as well as to create a "power-on reset" effect. This power-on reset effect is achieved with a simple three-resistor voltage divider with a capacitor across the middle resistor. Assuming the capacitor is discharged, the output of the circuit (which is the NMOS gate connection) will gradually change from a higher voltage to a lower voltage, meaning the circuit will always attempt to close the PMOS on startup. Otherwise, I theorized (though never experimentally verified) that the PMOS might never close at all, and our overcurrent protection would be so good at its job so as to never allow any current to flow. This was designed to work with both 24 V and 48 V supplies. Low-valued resistors were used to ensure that the turn-off time of the PMOS, given by the RC circuit of R16 and the gate capacitance, kept the switching loss within the device to within its dissipation rating. The below slide I created at the time of designing this circuit shows all the math involved to arrive at this answer.

Overcurrent Protection

- PMOS ([datasheet](#))
 - 93A, 100V blocking voltage, 20V max V_{gs}
- Calcs for switching loss
 - Must interrupt high current, but charging through R14 will mean that takes time
 - Assume linear I_d fall and V_{ds} rise, and that both take $3 \cdot R_g C_g$ to reach zero. This is conservative because V_{th}=1.5V (min) will be reached in $2 \cdot R_g C_g$.
 - C_g=13.5nF (max), R_g=330R, R_gC_g=4.5us, so $3 \cdot R_g C_g=15\mu\text{s}$
 - $E = \frac{1}{2} \cdot (V_{ds}=48V) \cdot I_d \cdot (t=15\mu\text{s}) = (0.36 \cdot I_d)\text{mJ}$; i.e. I_d=700A before hitting E_{max}=245mJ
 - This is just area of two triangles (V sloping up, I sloping down) multiplied
 - Another check: SOA (page 5) shows PMOS can withstand 100us pulse of V_{ds}=50V, I_d<200A.
 - As long as OC short is not severe (<0.25R), can be confident PMOS will be safe. Even if it is severe, it is very unlikely to hit the avalanche energy limit (E_{max}).



Math and justification for PMOS switching loss when overcurrent condition is met, given RC circuit created by R16 and the gate capacitance of the PMOS.

Power regulation

I made a design decision that might seem suspicious at first glance: all voltage regulation on the board is done with low-dropout linear regulators, or LDOs. This was done very simply because the power draw on the two required voltage lines was low enough that the simpler integration and less noisy output of an LDO won over the higher integration cost of a switching regulator. The tables below give conservatively high estimates for the current draw on each voltage line, using information from each device's datasheet; note that current draw from high-impedance voltage dividers and the like was considered negligible.

Current draw on 3.6 V line			
Device	Est. current (mA)	Quantity	Total current (mA)
Buffer	1	3	3
Inverter	1	3	3
Current sense amplifier	2.5	4	10
Zener diodes	1	4	4
Power LED	5	1	5
		Total:	25

Current draw on 5 V line			
Device	Est. current (mA)	Quantity	Total current (mA)

GaN half-bridge	5	3	15
3.6 V line draw	$3.6/5 * 25 = 18$	1	18
		Total:	33

Also note that the input current to an LDO is given by the product of the output current and V_{out}/V_{in} . My final selection for the two LDOs as well as some of their specifications are given in the table below.

Part	Datasheet	V_{in} range (V)	Max I_{out} (mA)
TLV70036DDCR (3.6 V)	Link	[2, 5.5]	200
MCP1792T-5002 (5 V)	Link	[4.5, 55]	100

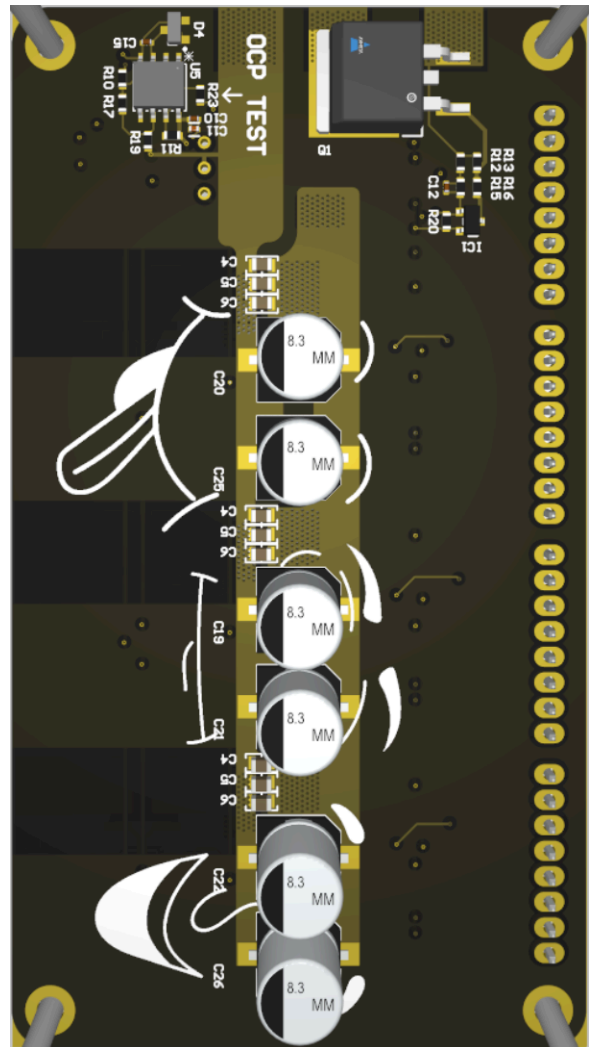
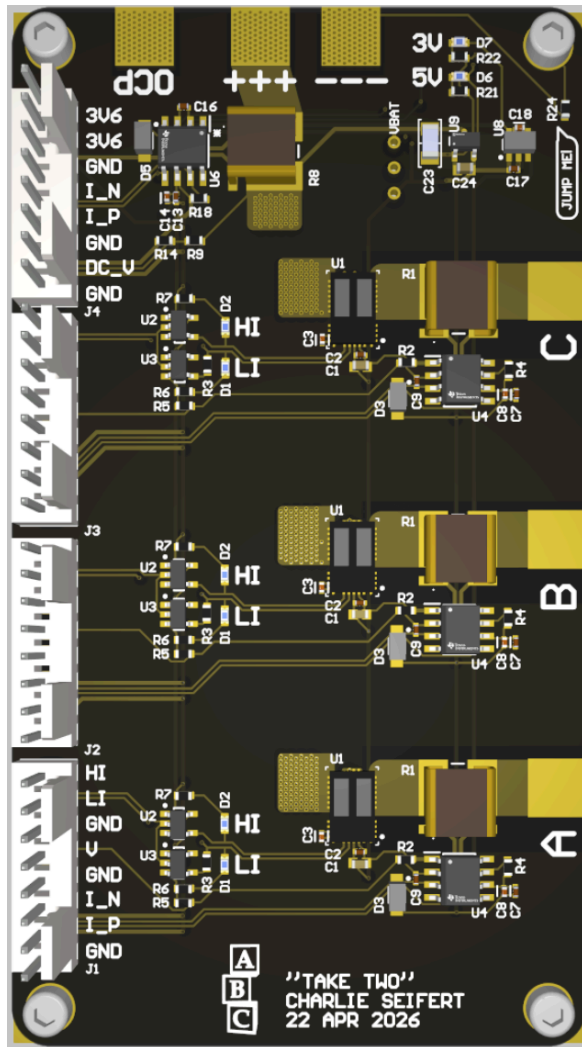
Note that the 5 V regulator feeds the 3.6 V regulator.

Just in case I was somehow very wrong in this analysis, I included three plated through-holes connected, spaced, and sized properly to allow the installation of a [TSR 2-2450](#), a fully-integrated 5 V DC-DC converter I had experience with, in parallel with the 5 V LDO. This device can output as high as 2 A, which I deemed more than enough.

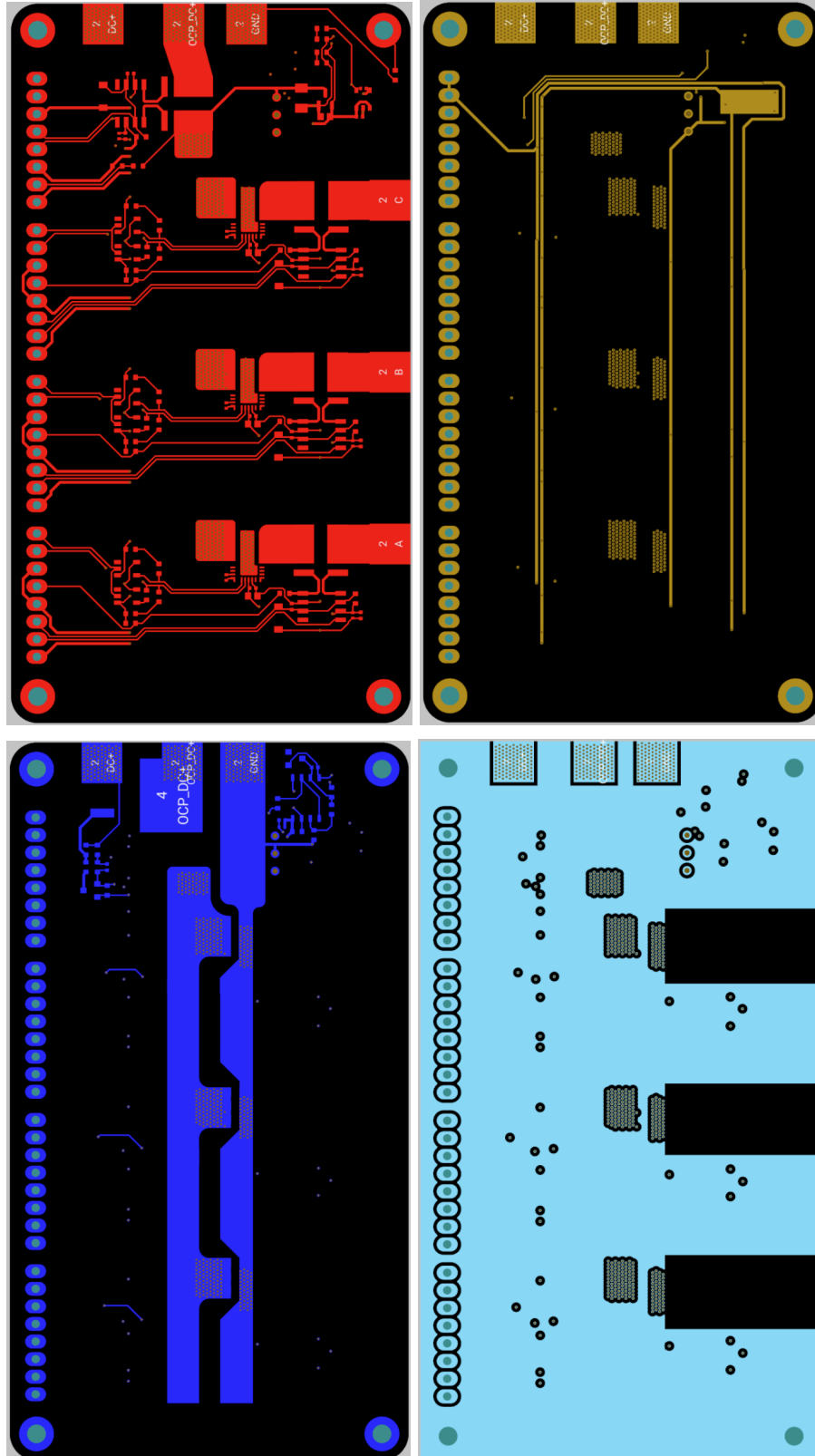
Layout

Pictures

The below figures show different views of the final layout.



3D render of top and bottom of assembled board from Altium Designer. Flipped the short way.



All four copper layers. Clockwise from top left: Top, Power, Shield, Bottom.

Goals

Following the requirements for this prototype, the two big priorities for the layout were being debug-friendly and maintaining signal integrity. The high-level decisions that each of these priorities led to are given below.

- Debug-friendly
 - Space out components
 - Large capacitors on bottom
 - Exposed vias for probing
 - Normal breadboard-style male headers
- Signal integrity for sensitive measurements
 - Differential routing of current sense lines, PWM input lines
 - Sensitive signals cross power planes perpendicularly
 - Shielding traces adjacent to sensitive traces
 - Shield layer

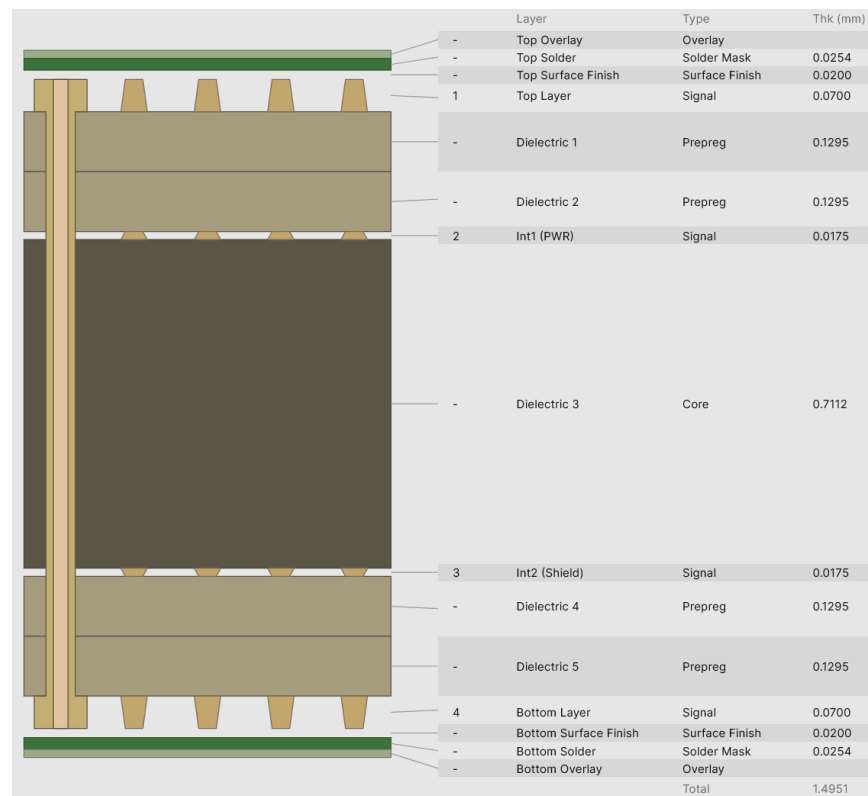
The previous year's board was too dense for a prototype. The initial assembly did not go smoothly, as a slightly heavy solder paste application caused shorts that were very difficult to debug due to the density. This year, I made it a priority to make the board more debug-friendly. It already helped that I only cared to control a single BLDC, not two like the previous project. The layout therefore only crams devices together if it is obviously beneficial for them to be so. This led to a layout strategy of clumping together devices only where needed, then using more real estate in connecting those clumps together, following the theory that the less dense the board is, the easier it is to debug. You can also see that even within clumps, any two devices are often not crammed super close together and maintain some breathing room to limit the odds of a short during assembly, and to allow for easy probing.

Noisy measurement signals would worsen the performance of certain control algorithms, making it harder to know if the algorithm is the problem and not just signal noise. This led me to give quite a bit of thought to how to route sensitive signal lines, particularly the current and voltage readings. The lines from the shunt to the amplifiers are a good example; any noise that couples into them would be amplified before going back to the MCU. Therefore these lines are very short. Differential routing for the post-amp current sense lines means that common-mode noise affects both signals simultaneously, and I laid out each phase connector to allow for ADC reading of both sides of the differential signal. Perpendicular crossing of these lines limits crosstalk by maintaining orthogonality between the two conductors' magnetic fields. The shield layer keeps the fast changing E-field present on the two long pours of the DC bus on the bottom layer from affecting the sensitive signal copper on the top.

Overview

The board has four layers. The high-power pours reside on the top and bottom layers, with the top holding the DC input and output phase pours, and the bottom holding the main strip of DC+ and DC-. This was done because the bulk capacitors reside on the bottom of the board, which was in

turn done to make the geometry more favorable and because it would make probing the top side of the board easier. The inner layers respectively carry the routing of power and ground, and a shielding layer. The full stackup information is shown in the figure below.

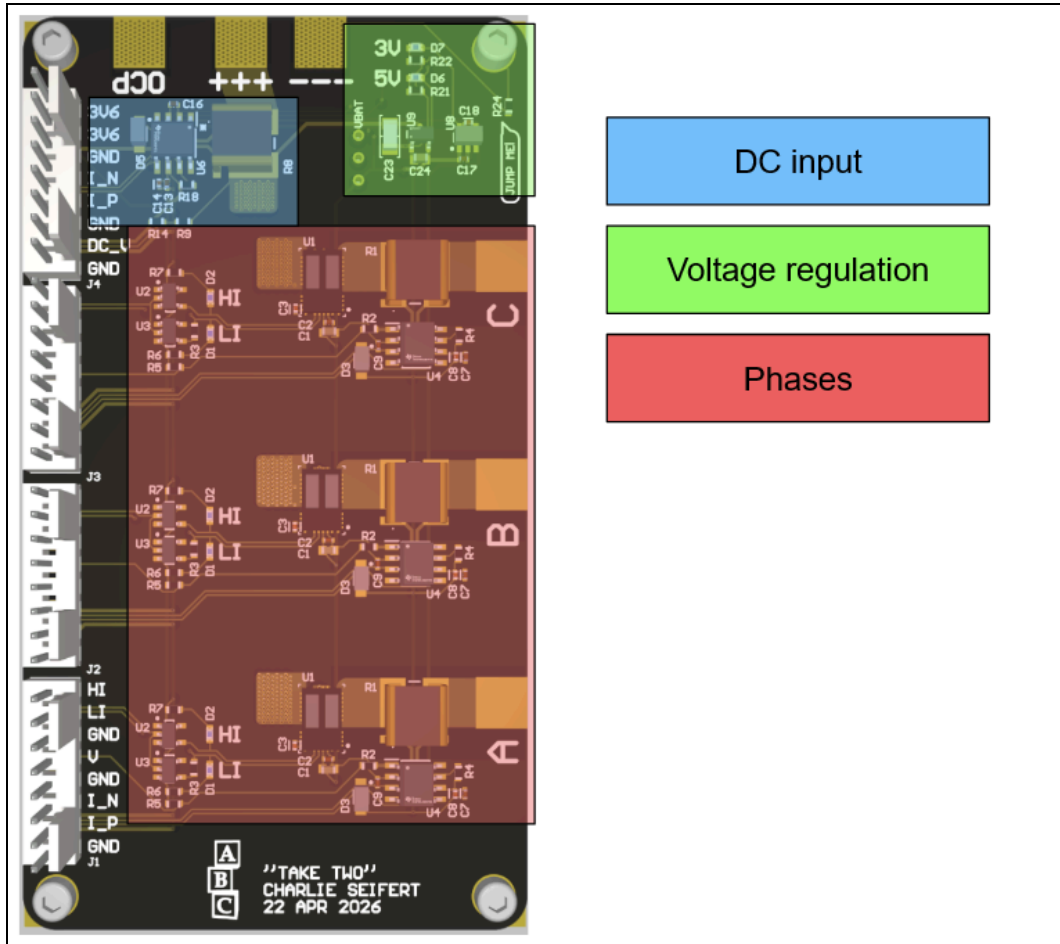


Layer stackup from Altium Designer. All of these are default options for a four-layer board, except for the thicker top and bottom copper layers, which were 2oz.

The board is organized into three main sections:

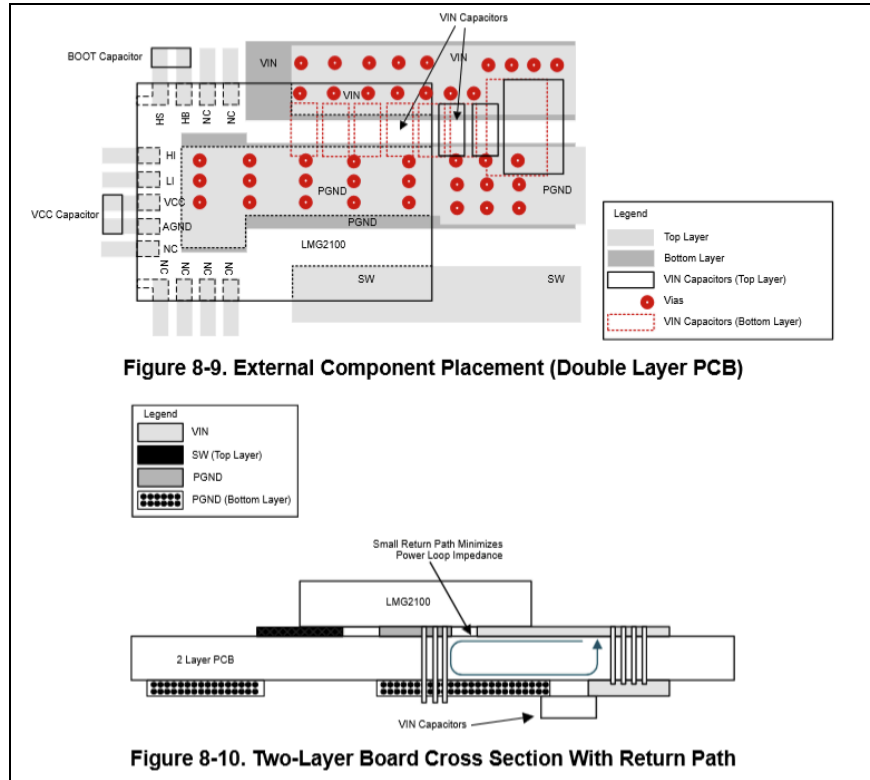
- Phases (3x: A B and C)
 - Input buffers
 - PWM indication LEDs
 - GaN chip and accompanying capacitors
 - Current sensing
- DC input
 - DC current sensing
 - OCP circuitry (bottom)
- Voltage regulation
 - 5 V and 3.6 V LDOs and accompanying capacitors
 - Power indication LEDs
 - Through-holes for optional 5 V DC-DC
 - Jumper to connect SHIELD to GND

These sections are indicated in the figure below.



Sections of the board layout.

Note the presence of an additional solder pad titled "OCP"; this pad is switched to the "+++" pad (itself accessible to allow the bypassing of OCP) via the PMOS that the OCP circuit opens when it detects overcurrent. In the single prototype I assembled, I did not install any of the OCP circuitry and connected the DC input across the "+++" and "---" pads, which I will refer to as DC+ and DC-. The DC- net immediately vias to the bottom layer, where it connects into a long pour that ties into the bulk capacitors for most of the length of the board. It also ties into the board's signal GND at a single point. The DC+ net goes through a shunt for DC current measurement, then vias to the bottom layer to connect into a similar pour for the bulk capacitors. Careful attention was paid to make sure the vias on these pads were untented on the top and tented on the bottom. The large electrolytic capacitors on the bottom are placed in pairs between the GaN chips on the top, allowing for large sections of stitching vias to be placed partially in the pad of the GaN chips to connect to the long DC+ and DC- pours. (This also presented an irresistible anthropomorphizing opportunity – their names are Craig, Bon, and Alan.) Note that all bulk capacitance is on the underside of the board. This adds some undesirable inductance to the power loop of the GaN chips, but it is not so bad, as evidenced by the TI-recommended layout strategy for a 2-layer board using the chips below.



Recommended layout strategy for bulk capacitance ("VIN Capacitors") for a 2-layer board.

The three phase layouts are basically identical. The buffer and inverter drive the inputs of the GaN chip, then copper pours connect the half-bridge chip to the shunt to the output solder pad. The current sense amplifier and accompanying passives are placed close to the shunt to minimize noise coupled into the amplifier inputs. Many stitching vias connect between the large copper pours carrying high currents.

The voltage regulation section is nothing special. A small GND pour underneath the LDOs is present to allow easy for easy vias. Routing of these power domains required a bit of deliberation, as it was going to be difficult to reach the current sensing circuitry with a clean power line (which could have been 5 V or 3.6 V, given the wide input supply range for the INA241) and signal GND, as well as GaN chip 5 V pins. Since the GaN chip definitely needed 5 V, and I could reach their pins relatively easily with a horizontal trace (perpendicular to the high-dI/dt outputs switch nodes),

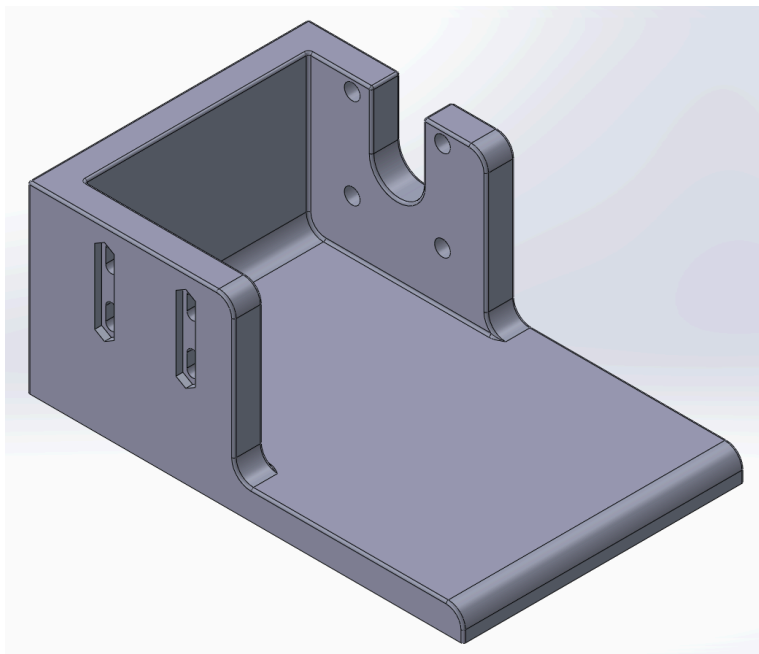
The board was manufacturing using JLCPCB. The manufacturing parameters for the board are given in the figure below.

Product Detail			
Gerber file:	apr23_1722_Y51	Build Time:	3-4 days
Base Material:	FR-4	Layers:	4
Dimension:	103.6 mm* 57.3 mm 103.63mm*57.28mm	PCB Qty:	10
Product Type:	Industrial/Consumer electronics	Different Design:	1
Delivery Format:	Single PCB	PCB Thickness:	1.6mm
Specify Stackup:	no No requirement	Layer Sequence:	
PCB Color:	Black	Silkscreen:	White
Material Type:	TG135	Via Covering:	Plugged
Surface Finish:	HASL (with lead)	Deburring/Edge rounding:	No
Outer Copper Weight:	2 oz	Inner Copper Weight:	0.5 oz
Gold Fingers:	No	Electrical Test:	Flying Probe Fully Test
Castellated Holes:	no	Press-Fit Hole:	No
Edge Plating:	No	Mark on PCB	Remove Mark
Blind Slot:	No	Min via hole size/ diameter:	0.3mm/(0.4/0.45mm)
Via Plating Method:	Not Specified	4-Wire Kelvin Test:	No
Paper between PCBs:	No	Appearance Quality:	IPC Class 2 Standard
Confirm Production file:	No	Silkscreen Technology:	Ink-jet Printing Silkscreen
Package Box:	With JLCPCB logo	Inspection Report:	No
Board Outline Tolerance:	±0.2mm(Regular)	UL Marking:	No
Countersink Hole:	No	Backdrill:	No
Humidity Indicator Card:	No	Shortage Preference:	Require Full Quantity

Manufacturing options selected for production at JLCPCB.

Mechanical design

The encoder and the diametrically-magnetized magnet attached to the rotor had to be rigidly co-located. A single, simple 3D print did the job. The CAD is shown in the screenshot below.



Motor mount CAD. File available [here](#).

It has the hole pattern for the motor's mounting holes and vertical slots for the encoder's, allowing vertical adjustment of the encoder. Recessed, wider slots on the outside of the encoder mounting slots made assembly a bit easier. The motor's hole pattern was incorrect, and I had to drill different holes - there was no official CAD for it and I must have measured the hole distance wrong. The mount was designed to have the wires from the motor leave out of the top and the motor viewable from the front, so it can be easily installed and viewed. Finally, a plate extends out from the front of the model, providing a nice surface to clamp the whole assembly securely to a table when testing. This design worked great: it was plenty thick to hold everything securely, the vertical adjustment of the encoder came in handy, and I didn't have to hold a small M2 nut due to the hexagonal slots. After assembly, the spacing between encoder and motor was a little tight, as I measured the thickness of an axially-magnetized magnet as opposed to a diametrically-magnetized magnet, but that is the only thing I would change.

Software

MCU selection and toolchain

One of the main things I was excited for with this project was to get experience with STM32 development. STM32 is a family of 32-bit microcontrollers from ST Microelectronics popular for its wide range of device capabilities, high performance, and ease of development. I had overseen a couple really awesome projects using STM32 MCUs during my time on Northwestern's Formula SAE team, and I was happy that this project presented an opportunity to get hands-on with one myself.

Buying an off-the-shelf development kit ("devkit") was a no-brainer for the goals of this project. ST calls their devkits Nucleo boards. I sought a Nucleo that used an STM32F4 chip, which is the highest-performance of their already high-performance F line. I had heard of it being used in other demanding motor control applications, and at the risk of sounding like a broken record, I had no intention of limiting performance of my board due to MCU selection if it was reasonable to acquire one. I settled on the STM32F446, for no reason greater than its availability and fast shipping on Amazon.

Another no-brainer was the use of ST's own STM32 development toolchain, which uses two applications: STM32CubeIDE ("CubeIDE") and STM32CubeMX ("CubeMX"). The former is the IDE where you spend most of your time writing C/C++ code and running the very capable debugger, and the latter is an extremely handy program that enables super simple graphical setup of programs requiring certain peripherals. These programs work together nicely to make any changes – whether low-level logic changes to the code or higher-level modification to peripheral setup – very simple.

Peripheral bringup

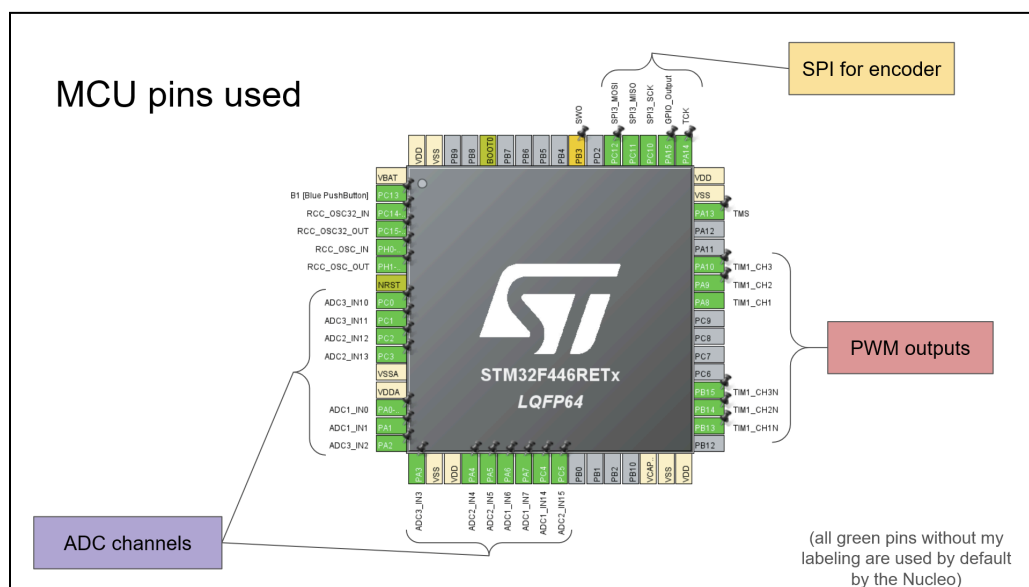
My general approach to development for this project was to heavily rely on the STM32F446's highly capable peripherals and minimize the complexity of code written by me. I anticipated that this strategy would make debugging the project simpler without limiting the capability of my programs,

as my goals did not include making highly sophisticated or advanced control algorithms. In particular, my STM32 MCU makes direct memory access (DMA), which enables peripherals nad memory to exchange data without the need to do so explicitly in the main loop, very simple to use, and it can be used with basically every peripheral I could ever find the need for. This also supports my goal of enabling fast control, as you don't have to spend as much CPU time doing variable assignments.

The basic peripherals that form the building blocks of the sort of simple-yet-advanced motor control program I was looking to write were:

- Analog-to-digital converter (ADC)
- Pulse-width modulation (PWM)
- Serial-peripheral interface (SPI)

The following sections detail my bringup process for each. The below figure serves as a visual reference for which pins each peripheral was assigned to on the STM32F446 64-pin package. Note that this pin layout was carefully decided to be able to be replicated on a custom PCB in the future.



Pin assignment for the three peripherals I used.

Analog-to-digital converter

My first goal with my new STM32 devkit was to investigate how the ADC peripheral worked. For my first actual program (beyond the basic blinky to validate the toolchain and board worked alright – they did), I ran a program that used just one channel of one ADC. Once I got that working, I wanted to take a step closer to how I would actually be using the ADC peripheral, with many channels being used. One good thing about the STM32F446 is its three independent ADC modules, which you could configure to simultaneously sample three separate voltages at once. This is explicitly advertised to be used in motor control algorithms, where you want to be sure that all current measurements are made at the same time because phase currents can change quickly. As shown in the pinout figure

above, I needed a lot of ADC readings. In groups of which should be simultaneously sampled, these were:

- Phase current sense measurement (3x)
- Phase current sense reference (3x)
- Phase voltage sense measurement (3x)
- Phase voltage sense reference (3x)
- DC current sense (1x)
- DC voltage sense (1x)

This totals 14 ADC readings I wanted to make, requiring 14 ADC channels configured through STM32CubeMX. I grouped them into groups of 3, sampled in the order given in the above bullets. I grouped DC current and voltage sense together, then threw in a repeated channel in with them to make an even 5 sampling events of 3 voltages. I configured DMA to place all 15 readings into a buffer called `adc_dma_buffer`.

Some subtleties in peripheral setup caused some hiccups initially. One was caused by the fact that I tried to do continuous conversion and then DMA requests after each. This would have been fine if not for CubeMX's insertion of an interrupt upon every DMA request. You can't configure this from CubeMX, and I honestly don't know why they do it, especially if you select continuous conversion – it basically defeats the purpose of using DMA. Once I removed the two lines shown in the figure below, though, DMA hummed along nicely in the background instead of interrupting the CPU every time it completed a transfer.

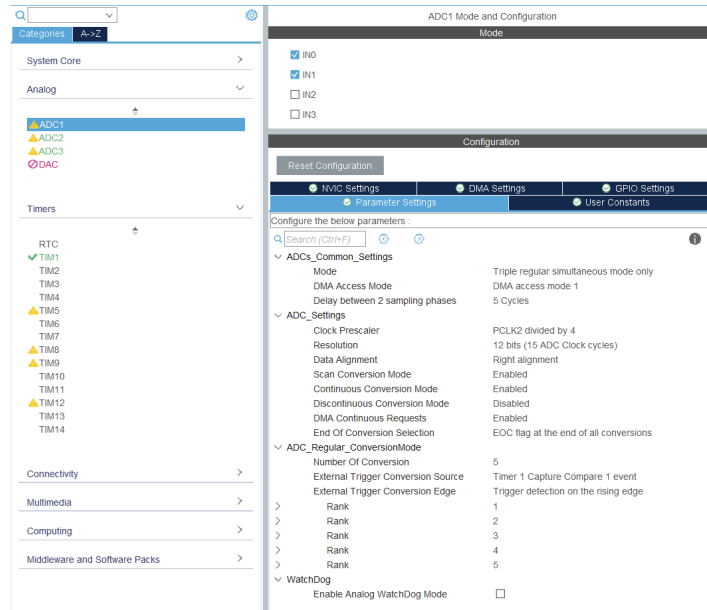
```
static void MX_DMA_Init(void)
{
    /* DMA controller clock enable */
    __HAL_RCC_DMA2_CLK_ENABLE();

    /* DMA interrupt init */
    /* DMA2 Stream0 IRQn interrupt configuration */
    HAL_NVIC_SetPriority(DMA2_Stream0_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(DMA2_Stream0_IRQn);
}
```

DMA interrupt inserted by CubeMX.

Note that these two lines will have to be commented out every time you generate code from CubeMX.

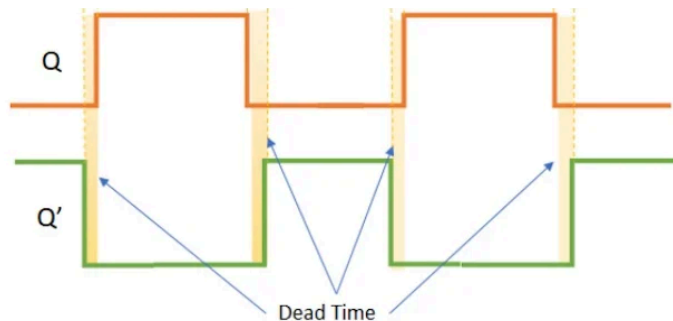
For triple-ADC operation, ADC1 works as a master and the other two work as slaves. See the figure below for the CubeMX setup I settled on for ADC1.



Prototype CubeMX setup for ADC1, which acts as master for ADC2 and ADC3.

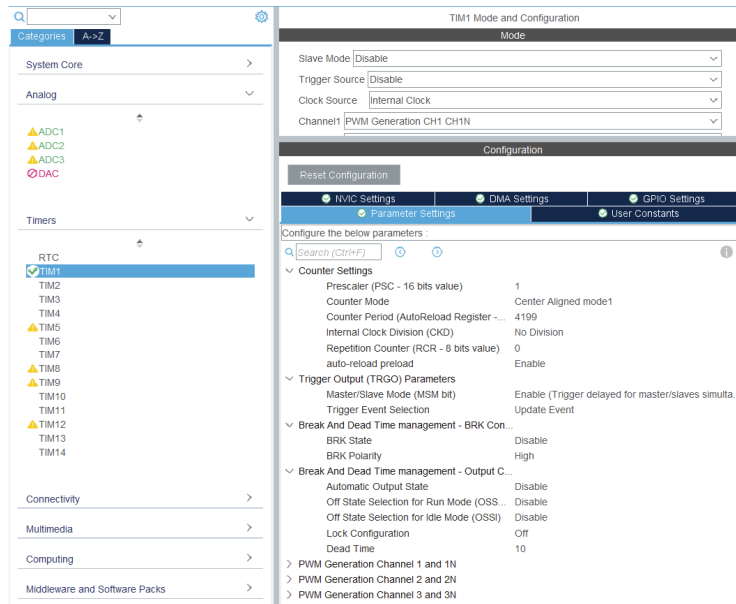
Pulse-width modulation

Pulse-width modulation is used to modulate current through a transistor while keeping it in its most efficient modes of operation – namely, not half-on. On the STM32F446, PWM capability is rolled into all generic timers, but more capability is present in the "Advanced timers", TIM1 and TIM8. These allow for complementary PWM outputs with inserted dead time, which is explicitly designed for half-bridge and motor control. The use of complementary PWM corresponds with the use of a 6PWM configuration on my custom board. The figure below shows the signals I wanted to produce using this peripheral.



Complementary PWM outputs with inserted dead time. [Source](#)

This turned out to be surprisingly easy to set up and use: simply decide on a total PWM period, do the rest of the setup, and then update the duty cycle of the channel by directly writing to the CCR field in the main loop. The CubeMX setup for PWM generation is shown below.



Prototype CubeMX setup for TIM1, which generates three complementary PWM channels.

With a prescalar of 1, a counter period of 4200 corresponds to a switching frequency of 1 kHz. This is super low for high-bandwidth control, but for the purposes of proving out the hardware, was perfectly fine. Center-aligned mode is very important for motor control as it leads to the proper overlapping of the phase PWM signals to produce torque. I am not exactly sure what would happen otherwise, but I am pretty sure it would at the very least be highly inefficient. A dead time of 1 us was used and is likely conservatively long. As a demonstration of how to update the duty cycle of each channel's PWM output, the figure below shows a snippet of my final position control code; see the last three lines where the CCRx fields are directly updated.

```
// phase vector magnitudes
magA = amp1 * cosf(angleV-angleA);
magB = amp1 * cosf(angleV-angleB);
magC = amp1 * cosf(angleV-angleC);

// assign duty cycles
dutyA = 0.5 + magA*0.5;
dutyB = 0.5 + magB*0.5;
dutyC = 0.5 + magC*0.5;
// *****

// assign duty cycles
TIM1->CCR1 = (int) (pwm_period * dutyA);
TIM1->CCR2 = (int) (pwm_period * dutyB);
TIM1->CCR3 = (int) (pwm_period * dutyC);
```

How to update the duty cycles of each PWM channel. Note dutyX values are in range [0,1].

With ADCs working and PWM set up, I could control an H-bridge to drive a DC motor at different speeds. A video of it in action is available [here](#).

Serial-peripheral interface

The last step to enable my first actual motor control program was to read motor position. For this I again relied on work done on the previous year's project and used the AS5147 on-axis magnetic SPI encoder IC to measure the angle of a diametrically-magnetized magnet mounted to the backside of a BLDC. Note that you don't need to know rotor position if you don't care to control rotor position, and you can instead use something like sensorless control which reads voltages off the motor phases to estimate rotor position. That wasn't a goal of this project so I opted to stay away from sensorless, at least for initial bringup.

Having used the AS5147 before, I knew its quirks. It exclusively uses 16-bit SPI transactions, but only sends 14-bit data, generating a parity bit for bit 14 and not even using bit 15 (the most significant bit). Simple masking did the trick for extracting the data I wanted, but the bigger challenge was getting the STM32 hardware abstraction layer (HAL) to read 16-bit data, as the main functions for SPI are meant for 8-bit data. The following figure shows the sort of casting required to use the HAL's pre-built SPI functions, as well as the masking required to extract the angle data from the encoder.

```
// calibrate encoder
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_15, GPIO_PIN_RESET);
HAL_Delay(0.01);
HAL_SPI_TransmitReceive(&hspi3, (uint8_t*)&encoder_reg, (uint8_t*)&angle_mes_encoder, 1, 100);
HAL_Delay(0.01);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_15, GPIO_PIN_SET);
HAL_Delay(0.01);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_15, GPIO_PIN_RESET);
HAL_Delay(0.01);
HAL_SPI_Receive(&hspi3, (uint8_t*)&angle_mes_encoder, 1, 100);
HAL_Delay(0.01);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_15, GPIO_PIN_SET);

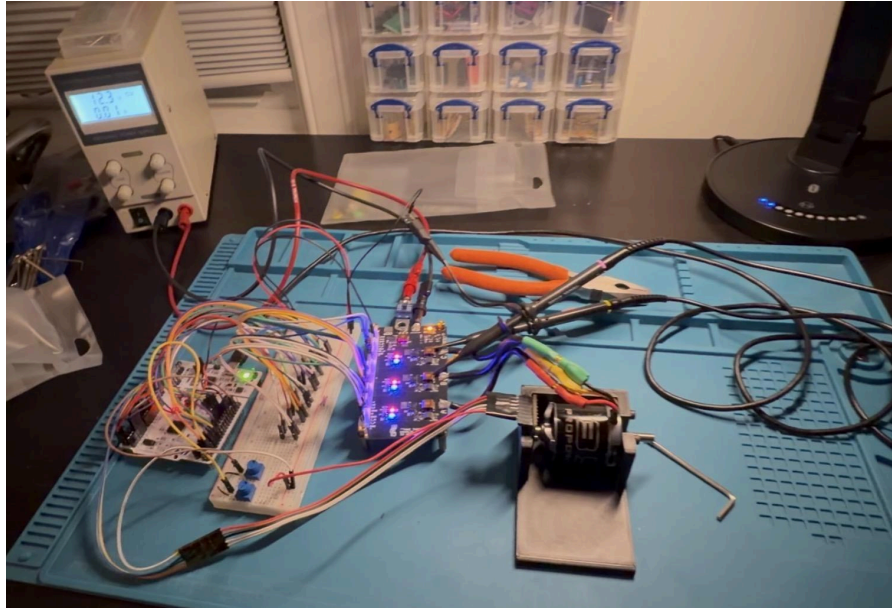
angle_mes_encoder &= 0x3FFF;
```

How to read angle data off of the AS5147 magnetic encoder IC via SPI.

In the end, this is a pretty simple part of the program. For pinout purposes, SPI3 was used. Luckily, the HAL doesn't make it too difficult to use any SPI block except for SPI1, like you might see with other microcontrollers or SPI libraries.

Results

At the time of writing on June 11 2026, I have concluded work on this project for the quarter to allow myself to finish this report. I reached were basic position control then velocity control of a low-power (700W) BLDC motor. While I had originally aimed to achieve more advanced control, I am happy with this result and how I spent my time – methodically testing and layering on complexity slowly has given me full confidence that the foundations of this project are rock solid and will allow me to continue work towards more advanced control. I am proud of the hardware setup I have concluded the quarter with, as it contains everything I will need to make more progress. It is shown in the photo below.



Hardware setup at the conclusion of this quarter.

The following sections detail the two modes of control I implemented.

Position control

See [this file](#) for the position control code. This program is the culmination of every previous piece of firmware I had written: it reads all of the board's ADC values via DMA, commands complementary PWM values on the positive and negative halves of three TIM1 channels, and reads from a SPI encoder to determine motor position. As of the time of writing, the program uses a blocking SPI call; a perfect next step to speed up the main loop is to take advantage of the STM32's SPI DMA. The motor the code is set up for is a [Propdrive v2 3542 1000KV BLDC](#), a 17V 47A outrunner motor with 14 poles. I found this in the robotics lab and liked its high current rating, as it would use most of the 53A limit of the GaN chip when it came to torque control. After basic peripheral setup and initialization, the code works as follows:

1. Calibrate motor
 1. Command some arbitrary, unbalanced duty cycle: Phase A is driven slightly positive, and phases B and C are driven slightly negative, bringing the motor angle to a stable position aligned with phase A
 2. Wait a short while as the rotor settles
 3. Read the encoder angle and assign it to the zero value, then return to balanced duty cycles to release the rotor
 4. Assign electrical phase angles
 1. Phase A angle = zero value
 2. Phase B angle = zero value + $2\pi/3$
 3. Phase C angle = zero value + $4\pi/3$
2. Position control loop

1. Read motor angle from encoder via 16-bit SPI transaction
2. Read potentiometer, assign to target angle
3. Calculate angle error, convert to electrical angle via motor pole count, then assign to amplitude of duty cycle via simple proportional controller
4. Calculate phase duty cycles
 1. Find desired stator voltage electrical angle with ortho(rotor angle) in the direction that would produce torque in the desired direction
 2. Take dot product of desired stator voltage electrical angle with rotor electrical angle
5. Write duty cycle values to PWM peripheral memory
6. Loop

There were several hiccups before I arrived at intuitive, spring-like behavior of the motor; it was through this process that I learned how to factor in electrical angle. The idea is that the stator coils repeat every so often as you move around the motor's mechanical angle range. Selecting the number of stator coils and rotor magnets is an important decision that determines many motor properties. The number of times they repeat is indicated by the motor's "pole count" as it's often given. This means that while a certain rotor mechanical angle corresponds to a single electrical angle, a certain electrical angle could correspond to (pole pairs) amount of mechanical angles. So what we do is map the mechanical angle to the electrical angle it corresponds to, then deal only in that domain to do the remainder of the math. Before I realized that, I was repeatedly stumped by the variety of different stable points of the motor's rotation not at the target angle, and its erratic behavior when applying external a torque to it. This behavior was due to the relationships between target encoder angle, measured encoder angle, and the output phase duty cycles being wonky due to the fact that I didn't understand how to factor in rotor electrical angle, wrap angle error readings (so 6.27 rads was considered close to 0 rads), or which value of pole count to use. The solutions to those second two problems were to subtract or add multiples of $\pi/2$ to bring the angle error within $[-\pi, \pi]$, then to use the number of pole *pairs*, not total pole count, which was was 7 for our motor.

Speed control

Speed control as I wrote it is very simple: simply add a certain amount to a target angle over time. So the system tracks a changing target angle. Pretty simple. See [this video](#) for a final demo, showcasing position and speed control.

Summary

The most impactful results include some of which I've already written about: the small things learned along the way that came from tackling subtle but often mighty problems. I went from messing up the setup of my first STM32CubeIDE project to stepping through compiled assembly and reading memory directly to determine the cause of a DMA problem; I started with no understanding of how bootstrapping a high-side FET works and ended up discovering organically why you can't indefinitely close it, solving an open question in my mind that had lasted since the bringup of the previous year's project; I went from knowing vaguely that the STM32 is "good for motor control" to

understanding and actually implementing code that took advantage of its features. Most of all, this project gave me greater confidence in my ability to design good solid hardware and solve all sorts of problems during bringup, in hardware or software.

Future work

The most obvious future work is to get vector control working. The good news is that all of the hardware is proven and much of the software basis is already written, and it should just be a matter of trying it and working through the bugs that pop up. The only big leap in the progression from position control to vector control is a more complex software control loop and the use of phase current readings, which are as yet untested at high current.

After that, any number of things could be the next steps:

- Speed up PWM, shorten dead time, and write as lightweight a control loop as possible to see how fast I can get it
- Find safe maximum board voltage, current, and power ratings
- Attempt sensorless, position-ignorant control using voltage measurements via high-frequency injection (HFI)
- Test overcurrent protection design
- Begin design of integrated driver board

I will use the findings from this project to design and develop a highly integrated motor drive, and I have full intention of continuing work on this project to get there.

Conclusion

Thank you to Professor Alan Sahakian for making this project possible. without the ability to get credit for this effort, it would not have happened. His vast experience of designing and seeing so many low-level circuits lent itself well to the process of designing the hardware as well as beginning to understand the STM32's peripherals, not to mention some great nerd conversations about measurement equipment and pacemakers. Thanks for being gracious even when I made you late to your dean meetings.

Sources

In conjunction with lessons from my hands-on discoveries, the following sources helped me learn so much this past quarter.

- TBarkin121 on Github: [GuardDog torque control code](#)
- Switchcraft: [Space Vector PWM Intro](#) (incredibly helpful!)
- Nidec: [Slot-pole combination](#)
- Teacher Sugiken's Motor Driver Dojo: [What is a Brushless Motor?](#)